Math	Pseudocode
$a^* = \arg max_a Q_k(a_i)$	`return argmax(exp_rewards)`

The following listing shows it as legitimate Python 3 code.

Listing 2.1. Finding the best actions given the expected rewards in Python 3

```
def get_best_action(actions):
    best_action = 0
    max_action_value = 0
    for i in range(len(actions)):                          1
        cur_action_value = get_action_value(actions[i])    2
        if cur_action_value > max_action_value:
            best_action = i
            max_action_value = cur_action_value
    return best_action
```

- ***1*** Loops through all possible actions
- ***2*** Gets the value of the current action

We use our above function $Q_k(a)$ on all the possible actions, and select the action that returns the maximum average reward. Since $Q_k(a)$ depends on a record of our previous actions and their associated rewards, this method will not evaluate actions that we haven't already explored. Thus, we might have previously tried levers #1 and #3 and noticed that lever #3 gives us a higher reward, but with this method we'll never think to try another lever, say #6, which, unbeknownst to us, actually gives out the highest average reward. This method of simply choosing the best lever that we know of so far is called a *greedy* (or exploitation) method.

2.2.2. Epsilon-greedy strategy

We need some exploration of other levers (other slot machines) to discover the true best action. One simple modification to our previous algorithm is to change it to an ε(epsilon)-greedy algorithm, such that with a probability, ε, we will choose an action, *a*, at random, and the rest of the time (probability 1 – ε) we will choose the best lever based on what we currently know from